

5 Advanced Next.js Problems and Solutions

1. Dynamic Routing with Catch-All Routes and Fallbacks

Scenario:

You need to build a documentation site like `/docs/intro`, `/docs/setup/environment`, etc., where routes are unknown at build time.

Solution:

- Use a catch-all route in ``pages/docs/[...slug].js``.

```
``js
// pages/docs/[...slug].js
import { useRouter } from 'next/router';

export default function DocPage({ content }) {
  const router = useRouter();

  if (router.isFallback) return <p>Loading...</p>;

  return (
    <div>
      <h1>Docs Page</h1>
      <pre>{JSON.stringify(content, null, 2)}</pre>
    </div>
  );
}

export async function getStaticPaths() {
  return {
    paths: [], // No paths pre-rendered at build
    fallback: 'blocking', // On-demand SSR
  };
}

export async function getStaticProps({ params }) {
```

```
const slugPath = params.slug.join('/');
const content = { path: slugPath, data: "Demo data" };

return {
  props: { content },
  revalidate: 60, // ISR
};
}
...

```

2. Preventing Hydration Mismatch

Scenario:

You render window or navigator values during SSR, causing hydration mismatches.

Solution:

- Use `useEffect` to run browser-only logic:

```
```js
import { useEffect, useState } from 'react';

export default function DeviceInfo() {
 const [isMobile, setIsMobile] = useState(false);

 useEffect(() => {
 setIsMobile(window.innerWidth < 768);
 }, []);

 return <p>{isMobile ? "Mobile View" : "Desktop View"}</p>;
}
...

```

## 3. Using Middleware for Role-Based Redirection

### *Scenario:*

---

You want to redirect unauthenticated users trying to access `/admin`.

### *Solution:*

---

- Use Next.js Middleware in `middleware.js`:

```
``js
// middleware.js
import { NextResponse } from 'next/server';

export function middleware(request) {
 const token = request.cookies.get('token')?.value;

 if (!token && request.nextUrl.pathname.startsWith('/admin')) {
 const url = request.nextUrl.clone();
 url.pathname = '/login';
 return NextResponse.redirect(url);
 }

 return NextResponse.next();
}

export const config = {
 matcher: ['/admin/:path*'],
};
``
```

## 4. Custom App Layouts with Per-Page Configuration

### *Scenario:*

---

You want different layouts for different pages (e.g., admin panel vs. public site).

### *Solution:*

---

- Wrap layout logic in `\_app.js`:

```
``js
// pages/_app.js
function MyApp({ Component, pageProps }) {
 const getLayout = Component.getLayout || ((page) => page);
```

```
 return getLayout(<Component {...pageProps} />);
 }
}
```

```
export default MyApp;
...
```

- In any page like `pages/admin/index.js`:

```
```js  
// pages/admin/index.js  
function AdminPage() {  
  return <div>Admin Dashboard</div>;  
}
```

```
AdminPage.getLayout = function getLayout(page) {  
  return (  
    <AdminLayout>  
      {page}  
    </AdminLayout>  
  );  
};
```

```
export default AdminPage;  
...
```

5. API Route Performance Optimization with Edge Functions

Scenario:

Your API route is latency-sensitive (e.g., geo IP check, auth check).

Solution:

- Use Edge API routes for faster response near the user:

```
```js  
// pages/api/geo.js
export const config = {
 runtime: 'edge', // Enables Edge Function
};
```

```
export default async function handler(req) {
 const country = req.geo?.country || 'unknown';
 return new Response(JSON.stringify({ country }), {
 headers: { 'Content-Type': 'application/json' },
 });
}
```